

Lab 04 Web Crawling & HTTP API Integration - Sample Code Guide

Purpose: This guide contains detailed sample code with explanations to help you understand the concepts before attempting the lab tasks.

How to Use This Guide:

1. Read through each sample carefully
2. Run the sample code on your machine
3. Study the explanations to understand how it works
4. Then attempt the corresponding tasks in Lab_07_Tasks.md

Table of Contents

1. Part 1: HTTP Basics - Simple Web Scraper
2. Part 2: Scraping Structured Data
3. Part 3: REST API Integration
4. Common Mistakes to Avoid
5. Quick Reference

Part 1: HTTP Basics - Simple Web Scraper



Sample Code: Understanding HTTP Requests and HTML Parsing

HTTP is the protocol that powers the web. Every time you visit a website, your browser sends HTTP requests and receives HTML responses.

Create `simple_scraper.go` :

```
1 | package main
```

```
2
3 import (
4     "bytes"
5     "fmt"
6     "io"
7     "net/http"
8     "strings"
9
10    "golang.org/x/net/html"
11 )
12
13 // Simple web scraper that fetches a page and extracts all links
14 func scrapeLinks(url string) ([]string, error) {
15     // Step 1: Make HTTP GET request
16     resp, err := http.Get(url)
17     if err != nil {
18         return nil, fmt.Errorf("failed to fetch page: %w", err)
19     }
20     defer resp.Body.Close()
21
22     // Check status code
23     if resp.StatusCode != http.StatusOK {
24         return nil, fmt.Errorf("bad status code: %d", resp.Stat
25     }
26
27     // Step 2: Read response body
28     body, err := io.ReadAll(resp.Body)
29     if err != nil {
30         return nil, fmt.Errorf("failed to read body: %w", err)
31     }
32
33     // Step 3: Parse HTML
34     doc, err := html.Parse(bytes.NewReader(body))
35     if err != nil {
36         return nil, fmt.Errorf("failed to parse HTML: %w", err)
37     }
38
39     // Step 4: Extract all links
40     links := extractLinks(doc)
41     return links, nil
42 }
43
44 // Recursive function to walk through HTML tree and find all <a> tags
45 func extractLinks(n *html.Node) []string {
46     var links []string
47
48     // Define recursive walker function
49     var walk func(*html.Node)
```

```

50     walk = func(n *html.Node) {
51         // If this is an <a> element
52         if n.Type == html.ElementNode && n.Data == "a" {
53             // Look for href attribute
54             for _, attr := range n.Attr {
55                 if attr.Key == "href" {
56                     links = append(links, attr.Val)
57                     break
58                 }
59             }
60         }
61
62         // Recursively walk through child nodes
63         for c := n.FirstChild; c != nil; c = c.NextSibling {
64             walk(c)
65         }
66     }
67
68     walk(n)
69     return links
70 }
71
72 // Extract text content from HTML node
73 func extractText(n *html.Node) string {
74     if n.Type == html.TextNode {
75         return strings.TrimSpace(n.Data)
76     }
77
78     var text string
79     for c := n.FirstChild; c != nil; c = c.NextSibling {
80         text += extractText(c)
81     }
82     return text
83 }
84
85 func main() {
86     // Test with a practice website
87     url := "http://quotes.toscrape.com/"
88
89     fmt.Printf("Scraping links from: %s\n", url)
90     links, err := scrapeLinks(url)
91     if err != nil {
92         fmt.Printf("Error: %v\n", err)
93         return
94     }
95
96     fmt.Printf("\nFound %d links:\n", len(links))
97     for i, link := range links {
98         // ...

```

```

98         if i < 10 { // Show first 10
99             fmt.Printf("  %d. %s\n", i+1, link)
100         }
101     }
102 }

```

Run:

```
go run simple_scraper.go
```

Expected Output:

```

Scraping links from: http://quotes.toscrape.com/
Found 15 links:
1. /
2. /login
3. /author/Albert-Einstein
4. /tag/change/page/1/
5. /tag/deep-thoughts/page/1/
...

```

**Detailed Code Explanation:****Understanding the scrapeLinks function:**

```

1 func scrapeLinks(url string) ([]string, error) {
2     // Step 1: Make HTTP GET request
3     resp, err := http.Get(url)

```

- `http.Get(url)` sends an HTTP GET request to the URL
- Returns `resp` (response object) and `err` (error if any)
- This is like opening a webpage in your browser

```

1     if err != nil {
2         return nil, fmt.Errorf("failed to fetch page: %w", err)
3     }
4     defer resp.Body.Close()

```

- Check if request failed (network error, DNS error, etc.)
- `defer resp.Body.Close()` ensures response body is closed when function exits
- **Critical:** Always close response bodies to prevent memory leaks

```
1     if resp.StatusCode != http.StatusOK {  
2         return nil, fmt.Errorf("bad status code: %d", resp.StatusCode)  
3     }
```

- Check HTTP status code (200 = OK, 404 = Not Found, 500 = Server Error)
- `http.StatusOK` is a constant equal to 200
- Always verify status before processing response

```
1     body, err := io.ReadAll(resp.Body)  
2     if err != nil {  
3         return nil, fmt.Errorf("failed to read body: %w", err)  
4     }
```

- Read entire response body into memory
- `body` is a byte slice containing the HTML content
- This is the raw HTML you see when you "View Page Source"

```
1     doc, err := html.Parse(bytes.NewReader(body))  
2     if err != nil {  
3         return nil, fmt.Errorf("failed to parse HTML: %w", err)  
4     }
```

- Parse HTML bytes into a tree structure (DOM tree)
- `bytes.NewReader(body)` creates a reader from the byte slice
- `doc` is the root node of the HTML tree

```
1     links := extractLinks(doc)  
2     return links, nil  
3 }
```

- Call helper function to extract links from the parsed tree
- Return the collected links

Understanding the `extractLinks` function:

```

1 func extractLinks(n *html.Node) []string {
2     var links []string
3
4     var walk func(*html.Node)

```

- Create a slice to store found links
- Declare `walk` as a recursive function variable
- Using function variable allows the function to call itself (recursion)

```

1     walk = func(n *html.Node) {
2         if n.Type == html.ElementNode && n.Data == "a" {

```

- Check if current node is an HTML element (not text, comment, etc.)
- Check if the element is an `<a>` tag (anchor/link)
- `n.Data` contains the tag name ("a", "div", "p", etc.)

```

1         for _, attr := range n.Attr {
2             if attr.Key == "href" {
3                 links = append(links, attr.Val)
4                 break
5             }
6         }

```

- Loop through all attributes of the `<a>` tag
- Look for the `href` attribute (contains the URL)
- `attr.Key` is the attribute name, `attr.Val` is its value
- Example: `<a href="/login">` → `Key="href", Val="/login"`
- Add the `href` value to our links slice

```

1      for c := n.FirstChild; c != nil; c = c.NextSibling {
2          walk(c)
3      }

```

- Iterate through all child nodes of current node
- `n.FirstChild` gets the first child
- `c.NextSibling` moves to the next sibling node
- Recursively call `walk` on each child (depth-first traversal)
- This explores the entire HTML tree

```

1      walk(n)
2      return links
3  }

```

- Start the recursive walk from root node
- Return all collected links

HTML Tree Structure Visualization:

```

html (root)
├── head
│   └── title
├── body
│   ├── div
│   │   ├── a (href="/login")    ← Found!
│   │   └── span
│   └── div
│       └── a (href="/about")    ← Found!

```

Key Concepts:

1. **HTTP Request/Response Cycle:** Client sends request → Server sends response
2. **HTML Parsing:** Converting text into tree structure
3. **Tree Traversal:** Visiting every node in the tree
4. **Recursion:** Function calling itself to process nested structures

5. Error Handling: Always check errors and handle them gracefully

Part 2: Scraping Structured Data

Sample Code: Extracting Quotes with Metadata

Create `quotes_scraper.go` :

```
1 package main
2
3 import (
4     "bytes"
5     "encoding/json"
6     "fmt"
7     "io"
8     "net/http"
9     "os"
10    "strings"
11
12    "golang.org/x/net/html"
13 )
14
15 // Quote represents a single quote with metadata
16 type Quote struct {
17     Text    string    `json:"text"`
18     Author  string    `json:"author"`
19     Tags    []string  `json:"tags"`
20 }
21
22 // Scrape quotes from quotes.toscrape.com
23 func scrapeQuotes(url string) ([]Quote, error) {
24     // Fetch page
25     resp, err := http.Get(url)
26     if err != nil {
27         return nil, err
28     }
29     defer resp.Body.Close()
30
31     body, err := io.ReadAll(resp.Body)
32     if err != nil {
33         return nil, err
34     }
35
36     // Parse HTML and extract quotes
37     doc, err := html.Parse(bytes.NewReader(body))
38     if err != nil {
39         return nil, err
40     }
41
42     // Traverse the DOM tree to find quote elements
43     var quotes []Quote
44     err = traverse(doc, &quotes)
45     if err != nil {
46         return nil, err
47     }
48
49     return quotes, nil
50 }
```



```

36         doc, err := ntml.Parse(bytes.NewReader(body))
37         if err != nil {
38             return nil, err
39         }
40
41         // Extract quotes
42         return extractQuotes(doc), nil
43     }
44
45     func extractQuotes(n *html.Node) []Quote {
46         var quotes []Quote
47
48         var walk func(*html.Node)
49         walk = func(n *html.Node) {
50             // Look for quote containers
51             if n.Type == html.ElementNode && n.Data == "div" {
52                 for _, attr := range n.Attr {
53                     if attr.Key == "class" && strings.Contai
54                         // Extract quote data
55                         quote := extractQuoteData(n)
56                         if quote.Text != "" {
57                             quotes = append(quotes, quote)
58                         }
59                         break
60                 }
61             }
62         }
63
64         for c := n.FirstChild; c != nil; c = c.NextSibling {
65             walk(c)
66         }
67     }
68
69     walk(n)
70     return quotes
71 }
72
73 func extractQuoteData(n *html.Node) Quote {
74     var quote Quote
75
76     var walk func(*html.Node)
77     walk = func(n *html.Node) {
78         if n.Type == html.ElementNode {
79             // Extract text
80             if n.Data == "span" {
81                 for _, attr := range n.Attr {
82                     if attr.Key == "class" && string
83                         quote.Text = strings.Trim
84                 }

```

```

85         }
86     }
87
88     // Extract author
89     if n.Data == "small" {
90         for _, attr := range n.Attr {
91             if attr.Key == "class" && string
92                 quote.Author = extractTe
93             }
94         }
95     }
96
97     // Extract tags
98     if n.Data == "a" {
99         for _, attr := range n.Attr {
100             if attr.Key == "class" && string
101                 tag := extractText(n)
102                 if tag != "" {
103                     quote.Tags = appe
104                 }
105             }
106         }
107     }
108 }
109
110     for c := n.FirstChild; c != nil; c = c.NextSibling {
111         walk(c)
112     }
113 }
114
115 walk(n)
116 return quote
117 }
118
119 func extractText(n *html.Node) string {
120     if n.Type == html.TextNode {
121         return strings.TrimSpace(n.Data)
122     }
123
124     var text string
125     for c := n.FirstChild; c != nil; c = c.NextSibling {
126         text += extractText(c)
127     }
128     return text
129 }
130
131 func main() {
132     url := "http://quotes.toscrape.com/"

```

```
133
134     fmt.Printf("Scraping quotes from: %s\n", url)
135     quotes, err := scrapeQuotes(url)
136     if err != nil {
137         fmt.Printf("Error: %v\n", err)
138         return
139     }
140
141     fmt.Printf("\nFound %d quotes\n\n", len(quotes))
142
143     // Display first 3 quotes
144     for i, quote := range quotes {
145         if i < 3 {
146             fmt.Printf("Quote %d:\n", i+1)
147             fmt.Printf("  Text: %s\n", quote.Text)
148             fmt.Printf("  Author: %s\n", quote.Author)
149             fmt.Printf("  Tags: %v\n\n", quote.Tags)
150         }
151     }
152
153     // Save to JSON
154     jsonData, err := json.MarshalIndent(quotes, "", " ")
155     if err != nil {
156         fmt.Printf("Error marshaling JSON: %v\n", err)
157         return
158     }
159
160     err = os.WriteFile("quotes.json", jsonData, 0644)
161     if err != nil {
162         fmt.Printf("Error writing file: %v\n", err)
163         return
164     }
165
166     fmt.Printf("Saved %d quotes to quotes.json\n", len(quotes))
167 }
```

Run:

```
go run quotes_scraper.go
```

Expected Output:

Scraping quotes from: `http://quotes.toscrape.com/`
Found 10 quotes

Quote 1:

Text: The world as we have created it is a process of our thinking...
Author: Albert Einstein
Tags: [change deep-thoughts thinking world]

Quote 2:

Text: It is our choices, Harry, that show what we truly are...
Author: J.K. Rowling
Tags: [abilities choices]

Saved 10 quotes to `quotes.json`



Detailed Code Explanation:

Understanding the Quote struct:

```
1 type Quote struct {  
2     Text    string    `json:"text"`  
3     Author  string    `json:"author"`  
4     Tags    []string  `json:"tags"`  
5 }
```

- Defines the structure of a quote with three fields
- `json:"..."` tags specify how field names appear in JSON
- When marshaled to JSON: `{"text": "...", "author": "...", "tags": [...]}`
- This struct represents one quote from the website

Understanding scrapeQuotes function:

```
1 func scrapeQuotes(url string) ([]Quote, error) {  
2     resp, err := http.Get(url)  
3     if err != nil {  
4         return nil, err  
5     }  
6     defer resp.Body.Close()
```

- Same HTTP request pattern as before
- Returns a slice of Quote structs instead of strings
- Always defer closing the response body

```

1      body, err := io.ReadAll(resp.Body)
2      if err != nil {
3          return nil, err
4      }
5
6      doc, err := html.Parse(bytes.NewReader(body))
7      if err != nil {
8          return nil, err
9      }

```

- Read and parse HTML - same pattern as simple scraper
- doc contains the parsed HTML tree

```

1      return extractQuotes(doc), nil
2  }

```

- Call helper function to extract quote data
- Return slice of Quote structs

Understanding extractQuotes function:

```

1  func extractQuotes(n *html.Node) []Quote {
2      var quotes []Quote
3
4      var walk func(*html.Node)
5      walk = func(n *html.Node) {

```

- Create empty slice to store quotes
- Define recursive walker function

```

1      if n.Type == html.ElementNode && n.Data == "div" {

```

- Look for `<div>` elements (quote containers)
- Not all divs contain quotes - need to check class name

```
1         for _, attr := range n.Attr {
2             if attr.Key == "class" && strings.Contains(attr.Val, "q
```

- Check if div has class attribute
- `strings.Contains` checks if class value contains "quote"
- Example: `<div class="quote">` or `<div class="quote featured">`

```
1             quote := extractQuoteData(n)
2             if quote.Text != "" {
3                 quotes = append(quotes, quote)
4             }
5             break
```

- Found a quote container! Extract its data
- Only add if quote has text (not empty)
- `break` exits the attribute loop (found what we need)

```
1         for c := n.FirstChild; c != nil; c = c.NextSibling {
2             walk(c)
3         }
```

- Continue walking through all child nodes
- Recursive depth-first traversal

Understanding `extractQuoteData` function:

```
1 func extractQuoteData(n *html.Node) Quote {
2     var quote Quote
3
4     var walk func(*html.Node)
5     walk = func(n *html.Node) {
```

- Create empty `Quote` struct to fill

- Another recursive walker for the quote container

```

1         if n.Type == html.ElementNode {
2             if n.Data == "span" {
3                 for _, attr := range n.Attr {
4                     if attr.Key == "class" && strings.Contains(attr.Val
5                         quote.Text = strings.Trim(extractText(n), ``)

```

- Look for `<span class="text">` containing quote text
- Extract text content using helper function
- `strings.Trim(text, " ")` removes quote marks from both ends
- Example: "Hello World" → Hello World

HTML Structure Being Parsed:

```

1 <div class="quote">
2     <span class="text">"The quote text here"</span>
3     <small class="author">Albert Einstein</small>
4     <div class="tags">
5         <a class="tag">change</a>
6         <a class="tag">thinking</a>
7     </div>
8 </div>

```

```

1         if n.Data == "small" {
2             for _, attr := range n.Attr {
3                 if attr.Key == "class" && strings.Contains(attr.Val
4                     quote.Author = extractText(n)

```

- Look for `<small class="author">` containing author name
- Extract text content without modification

```

1         if n.Data == "a" {
2             for _, attr := range n.Attr {
3                 if attr.Key == "class" && strings.Contains(attr.Val
4                     tag := extractText(n)
5                     if tag != "" {
6                         quote.Tags = append(quote.Tags, tag)

```

- Look for `<a class="tag">` elements
- Each tag link contains one tag
- Append each tag to the Tags slice
- Multiple tags per quote → slice grows

```

1         for c := n.FirstChild; c != nil; c = c.NextSibling {
2             walk(c)
3         }

```

- Recursively walk through quote container's children
- Ensures we find all nested elements

Understanding extractText function:

```

1 func extractText(n *html.Node) string {
2     if n.Type == html.TextNode {
3         return strings.TrimSpace(n.Data)
4     }

```

- Base case: if node is text, return its content
- `strings.TrimSpace` removes leading/trailing whitespace
- Example: `" Hello "` → `"Hello"`

```

1     var text string
2     for c := n.FirstChild; c != nil; c = c.NextSibling {
3         text += extractText(c)
4     }
5     return text
6 }

```


- Recursive case: concatenate text from all children
- Handles nested elements: `<span>Hello <b>World</b></span>` → "Hello World"

Understanding the main function:

```

1 func main() {
2     url := "http://quotes.toscrape.com/"
3
4     fmt.Printf("Scraping quotes from: %s\n", url)
5     quotes, err := scrapeQuotes(url)
6     if err != nil {
7         fmt.Printf("Error: %v\n", err)
8         return
9     }

```

- Define target URL
- Call scraping function
- Handle any errors

```

1     fmt.Printf("\nFound %d quotes\n\n", len(quotes))
2
3     for i, quote := range quotes {
4         if i < 3 {
5             fmt.Printf("Quote %d:\n", i+1)
6             fmt.Printf("  Text: %s\n", quote.Text)
7             fmt.Printf("  Author: %s\n", quote.Author)
8             fmt.Printf("  Tags: %v\n\n", quote.Tags)
9         }
10    }

```

- Display total count
- Loop through quotes with index
- `range` returns index and value
- Only show first 3 for brevity
- `%v` formats the slice nicely

```

1 |     jsonData, err := json.MarshalIndent(quotes, "", "  ")
2 |     if err != nil {
3 |         fmt.Printf("Error marshaling JSON: %v\n", err)
4 |         return
5 |     }

```

- Convert quotes slice to JSON
- `MarshalIndent` creates pretty-printed JSON
- First `""` is prefix (none), `" "` is indent (2 spaces)
- Result: nicely formatted JSON with proper indentation

```

1 |     err = os.WriteFile("quotes.json", jsonData, 0644)
2 |     if err != nil {
3 |         fmt.Printf("Error writing file: %v\n", err)
4 |         return
5 |     }

```

- Write JSON data to file
- `0644` is file permission (read/write for owner, read for others)
- File will contain the formatted JSON

```

1 |     fmt.Printf("Saved %d quotes to quotes.json\n", len(quotes))
2 | }

```

- Confirm successful save

Data Flow:

HTTP Request → HTML Response → Parse HTML → Walk Tree →
 Find Quote Divs → Extract Text/Author/Tags → Create Quote Struct →
 Collect All Quotes → Marshal to JSON → Save to File

Key Concepts:

1. **Struct Tags:** Control JSON serialization behavior
2. **Nested Extraction:** Walking tree within tree (quote container)

3. **Text Accumulation:** Combining text from multiple text nodes
4. **JSON Marshaling:** Converting Go structs to JSON format
5. **File I/O:** Writing data to disk
6. **String Manipulation:** Trimming, checking substrings

Part 3: REST API Integration

Sample Code: Working with JSON APIs

Modern applications get data from APIs. Let's learn how to make API requests and process JSON responses.

Create `api_client.go` :

```
1  package main
2
3  import (
4      "bytes"
5      "encoding/json"
6      "fmt"
7      "io"
8      "net/http"
9      "time"
10 )
11
12 // JSONPlaceholder Post structure
13 type Post struct {
14     UserID int    `json:"userId"`
15     ID      int    `json:"id"`
16     Title   string `json:"title"`
17     Body    string `json:"body"`
18 }
19
20 // Fetch posts from JSONPlaceholder API
21 func fetchPosts() ([]Post, error) {
22     url := "https://jsonplaceholder.typicode.com/posts"
23
24     // Create HTTP client with timeout
25     client := &http.Client{
26         Timeout: 10 * time.Second,
27     }
```

```
27     },
28
29     // Make GET request
30     resp, err := client.Get(url)
31     if err != nil {
32         return nil, fmt.Errorf("request failed: %w", err)
33     }
34     defer resp.Body.Close()
35
36     // Check status code
37     if resp.StatusCode != http.StatusOK {
38         return nil, fmt.Errorf("bad status: %d", resp.StatusCode)
39     }
40
41     // Read response body
42     body, err := io.ReadAll(resp.Body)
43     if err != nil {
44         return nil, fmt.Errorf("read failed: %w", err)
45     }
46
47     // Parse JSON
48     var posts []Post
49     err = json.Unmarshal(body, &posts)
50     if err != nil {
51         return nil, fmt.Errorf("json parse failed: %w", err)
52     }
53
54     return posts, nil
55 }
56
57 // Create a new post
58 func createPost(post Post) (*Post, error) {
59     url := "https://jsonplaceholder.typicode.com/posts"
60
61     // Convert post to JSON
62     jsonData, err := json.Marshal(post)
63     if err != nil {
64         return nil, err
65     }
66
67     // Create POST request
68     resp, err := http.Post(url, "application/json", bytes.NewBuffer(jsonData))
69     if err != nil {
70         return nil, err
71     }
72     defer resp.Body.Close()
73
74     // Parse response
75     var createdPost Post
```

```
76         err = json.NewDecoder(resp.Body).Decode(&createdPost)
77         if err != nil {
78             return nil, err
79         }
80
81         return &createdPost, nil
82     }
83
84     func main() {
85         // Fetch all posts
86         fmt.Println("Fetching posts from API...")
87         posts, err := fetchPosts()
88         if err != nil {
89             fmt.Printf("Error: %v\n", err)
90             return
91         }
92
93         fmt.Printf("Fetched %d posts\n\n", len(posts))
94
95         // Display first 3
96         for i, post := range posts {
97             if i < 3 {
98                 fmt.Printf("Post %d:\n", post.ID)
99                 fmt.Printf("  Title: %s\n", post.Title)
100                 fmt.Printf("  Body: %s\n\n", post.Body[:50]+"..")
101             }
102         }
103
104         // Create new post
105         newPost := Post{
106             UserID: 1,
107             Title:   "My New Post",
108             Body:    "This is a test post created via API",
109         }
110
111         fmt.Println("Creating new post...")
112         created, err := createPost(newPost)
113         if err != nil {
114             fmt.Printf("Error: %v\n", err)
115             return
116         }
117
118         fmt.Printf("Created post with ID: %d\n", created.ID)
119     }
```

Run:

```
go run api_client.go
```



Detailed Code Explanation:

Understanding the Post struct:

```
1 type Post struct {
2     UserID int    `json:"userId"`
3     ID      int    `json:"id"`
4     Title   string `json:"title"`
5     Body    string `json:"body"`
6 }
```

- Defines structure matching JSONPlaceholder API response
- Field names match JSON keys (case-sensitive!)
- `json:"userId"` maps Go field `UserID` to JSON key `userId`
- This is the contract between your code and the API

Understanding fetchPosts function:

```
1 func fetchPosts() ([]Post, error) {
2     url := "https://jsonplaceholder.typicode.com/posts"
```

- Define API endpoint URL
- This is a free fake API for testing

```
    client := &http.Client{
        Timeout: 10 * time.Second,
    }
```

- Create custom HTTP client with timeout
- `&http.Client{}` creates pointer to client struct
- `Timeout: 10 * time.Second` means request fails after 10 seconds
- **Important:** Always set timeout to prevent hanging forever

```
1 resp, err := client.Get(url)
2 if err != nil {
3     return nil, fmt.Errorf("request failed: %w", err)
4 }
5 defer resp.Body.Close()
```

- Make GET request using custom client
- %w wraps error (preserves error chain for debugging)
- Always defer closing response body

```
1 if resp.StatusCode != http.StatusOK {
2     return nil, fmt.Errorf("bad status: %d", resp.StatusCode)
3 }
```

- Check for 200 OK status
- API might return 404, 500, etc. - need to handle these

```
1 body, err := io.ReadAll(resp.Body)
2 if err != nil {
3     return nil, fmt.Errorf("read failed: %w", err)
4 }
```

- Read entire response into memory
- body contains raw JSON bytes

```
1 var posts []Post
2 err = json.Unmarshal(body, &posts)
3 if err != nil {
4     return nil, fmt.Errorf("json parse failed: %w", err)
5 }
```

- Parse JSON into Go struct
- &posts is pointer to slice - function fills it
- json.Unmarshal reads JSON and populates struct fields
- Field names must match JSON keys (or use struct tags)

JSON Response Example:

```
1  [
2    {
3      "userId": 1,
4      "id": 1,
5      "title": "sunt aut facere...",
6      "body": "quia et suscipit..."
7    },
8    {
9      "userId": 1,
10     "id": 2,
11     "title": "qui est esse",
12     "body": "est rerum tempore..."
13   }
14 ]
```

After Unmarshal:

```
1  posts[0].UserID = 1
2  posts[0].ID = 1
3  posts[0].Title = "sunt aut facere..."
4  posts[0].Body = "quia et suscipit..."
```

```
1      return posts, nil
2  }
```

- Return parsed posts slice

Understanding createPost function:

```
1  func createPost(post Post) (*Post, error) {
2      url := "https://jsonplaceholder.typicode.com/posts"
```

- Same endpoint, but we'll use POST method


```
jsonData, err := json.Marshal(post)
if err != nil {
    return nil, err
}
```

- Convert Go struct to JSON
- `json.Marshal` opposite of `Unmarshal`
- Result: `{"userId":1,"id":0,"title":"My Post","body":"..."}`

```
1 resp, err := http.Post(url, "application/json", bytes.NewBuffer(jso
2 if err != nil {
3     return nil, err
4 }
5 defer resp.Body.Close()
```

- Make POST request with JSON data
- `"application/json"` tells server we're sending JSON
- `bytes.NewBuffer(jsonData)` creates reader from bytes
- Server reads this as request body

Request Flow:

```
POST /posts HTTP/1.1
Host: jsonplaceholder.typicode.com
Content-Type: application/json
```

```
{"userId":1,"title":"My Post","body":"Test"}
```

```
1 var createdPost Post
2 err = json.NewDecoder(resp.Body).Decode(&createdPost)
3 if err != nil {
4     return nil, err
5 }
```

- Parse response directly from reader
- `NewDecoder` creates decoder that reads from stream

- More efficient than ReadAll + Unmarshal for large responses
- `Decode(&createdPost)` fills struct from JSON response

```
1 |     return &createdPost, nil
2 | }
```

- Return pointer to created post
- Server usually returns the created object with assigned ID

HTTP Methods Comparison:

Method	Purpose	Has Body	Idempotent
GET	Retrieve data	No	Yes
POST	Create new	Yes	No
PUT	Update/Replace	Yes	Yes
DELETE	Remove	No	Yes

Key Concepts:

1. **HTTP Client Configuration:** Timeouts prevent hanging
2. **Request Methods:** GET vs POST, when to use each
3. **Content-Type Header:** Tells server what format you're sending
4. **JSON Marshaling:** Go struct → JSON bytes
5. **JSON Unmarshaling:** JSON bytes → Go struct
6. **Streaming Decode:** More efficient for large responses
7. **Error Wrapping:** `%w` preserves error chain for debugging

Common API Patterns:

- `GET /resource` → List all
- `GET /resource/:id` → Get specific one

- POST /resource → Create new
- PUT /resource/:id → Update existing
- DELETE /resource/:id → Delete

Best Practices:

1. Always set timeout on HTTP client
2. Always close response body
3. Check status code before parsing
4. Use appropriate HTTP method
5. Set correct Content-Type header
6. Handle errors at each step

Common Mistakes to Avoid

✗ Mistake 1: Not Handling HTTP Errors

```
1 // Wrong - ignores status code
2 resp, _ := http.Get(url)
3 body, _ := io.ReadAll(resp.Body)
```

✓ Solution: Always check status

```
1 resp, err := http.Get(url)
2 if err != nil {
3     return nil, err
4 }
5 defer resp.Body.Close()
6
7 if resp.StatusCode != http.StatusOK {
8     return nil, fmt.Errorf("bad status: %d", resp.StatusCode)
9 }
```

✗ Mistake 2: No Rate Limiting

```
1 // Wrong - hammers the server
2 for i := 0; i < 100; i++ {
3     scrapeBooks(fmt.Sprintf("page-%d.html", i))
4 }
```

✅ Solution: Add delays

```
1 for i := 0; i < 100; i++ {
2     scrapeBooks(fmt.Sprintf("page-%d.html", i))
3     time.Sleep(1 * time.Second) // Be nice to servers!
4 }
```

❌ Mistake 3: Not Closing Response Body

```
1 // Wrong - resource leak
2 resp, _ := http.Get(url)
3 io.ReadAll(resp.Body)
4 // Forgot to close!
```

✅ Solution: Always defer close

```
1 resp, err := http.Get(url)
2 if err != nil {
3     return err
4 }
5 defer resp.Body.Close() // Always clean up
```

❌ Mistake 4: Ignoring nil Pointers

```
1 // Wrong - will panic if node is nil
2 title := node.FirstChild.Data
```

✅ Solution: Check for nil

```
1 | if node != nil && node.FirstChild != nil {  
2 |     title := node.FirstChild.Data  
3 | }
```

Quick Reference

HTTP Client Operations

```
1 | // Simple GET request  
2 | resp, err := http.Get("https://api.example.com/data")  
3 | defer resp.Body.Close()  
4 |  
5 | // GET with custom client  
6 | client := &http.Client{Timeout: 10 * time.Second}  
7 | resp, err := client.Get(url)  
8 |  
9 | // POST request  
10 | jsonData := []byte(`{"key": "value"}`)  
11 | resp, err := http.Post(url, "application/json", bytes.NewBuffer(jsonDat  
12 |  
13 | // Custom request with headers  
14 | req, err := http.NewRequest("GET", url, nil)  
15 | req.Header.Set("User-Agent", "MyBot/1.0")  
16 | req.Header.Set("Authorization", "Bearer token")  
17 | resp, err := client.Do(req)
```

JSON Operations

```
1 // Unmarshal JSON to struct
2 var data MyStruct
3 err := json.Unmarshal(jsonBytes, &data)
4
5 // Marshal struct to JSON
6 jsonBytes, err := json.Marshal(data)
7
8 // Pretty print JSON
9 jsonBytes, err := json.MarshalIndent(data, "", " ")
10
11 // Save JSON to file
12 err := os.WriteFile("data.json", jsonBytes, 0644)
13
14 // Load JSON from file
15 jsonBytes, err := os.ReadFile("data.json")
16 json.Unmarshal(jsonBytes, &data)
```

HTML Parsing

```
1 // Parse HTML
2 doc, err := html.Parse(strings.NewReader(htmlContent))
3
4 // Walk through nodes
5 var walk func(*html.Node)
6 walk = func(n *html.Node) {
7     if n.Type == html.ElementNode && n.Data == "a" {
8         // Found an <a> tag
9         for _, attr := range n.Attr {
10             if attr.Key == "href" {
11                 fmt.Println(attr.Val)
12             }
13         }
14     }
15     for c := n.FirstChild; c != nil; c = c.NextSibling {
16         walk(c)
17     }
18 }
19 walk(doc)
20
21 // Extract text
22 func extractText(n *html.Node) string {
23     if n.Type == html.TextNode {
24         return n.Data
25     }
26     var text string
27     for c := n.FirstChild; c != nil; c = c.NextSibling {
28         text += extractText(c)
29     }
30     return text
31 }
```

Web Scraping Ethics & Best Practices

DO:

- Read and respect robots.txt
- Add rate limiting (1-2 seconds between requests)
- Use descriptive User-Agent header
- Only scrape publicly available data

- Cache data to minimize requests
- Handle errors gracefully

❌ DON'T:

- Overload servers with rapid requests
- Ignore rate limits or HTTP 429 responses
- Scrape personal or private data
- Bypass authentication or paywalls
- Ignore Terms of Service
- Resell scraped data without permission

Next Steps

Now that you've studied the sample code:

1.  Make sure you understand each sample
2.  Run the samples on your machine
3.  Experiment with modifying the samples
4.  **Now attempt the tasks in Lab_04_Tasks**

Good luck! 

End of Lab 04 Sample Guide